

TECHNICKÁ UNIVERZITA V LIBERCI

Fakulta mechatroniky, informatiky a mezioborových studií

Ústav NTI

Školní rok: 2025/26

ZADÁNÍ SEMESTRÁLNÍHO PROJEKTU

Název projektu: Vývoj algoritmu zarovnávání optických elementů s využitím obrazových dat ze zaměřovací kamery interferometru **Řešitel(-é): Ondřej Donát**

Studijní obor: Aplikovaná informatika

Vedoucí projektu: Ing. Marek Stašík, Ph.D.

Zadání:

1. Seznámení se se základními principy interferometrie a řešení softwarových prostředků a algoritmů pro zpracování obrazu ze zarovnávací kamery interferometru. 2. Navrhnout architekturu softwaru v jazyce C# a zvolit vhodné softwarové prostředky a knihovny (např. OpenCV nebo Intel Integrated Performance Primitives) pro vyhodnocení obrazových dat ze zaměřovací kamery interferometru.
3. Implementace algoritmu pro segmentaci obrazu a detekci těžiště bodů.
4. Hodnocení dosažených výsledků.

Doporučená literatura:

- [1] PLÍVA, Z., J. DRÁBKOVÁ, J. KOPRNICKÝ a L. PETRŽÍLKA. *Metodika zpracování bakalářských a diplomových prací*. 3. upravené vydání. Liberec: Technická univerzita v Liberci, FM, 2019. ISBN 978-80-7494-455-0. Dostupné z: doi:10.15240/tul/002/978-80-7494-455-0.
- [2] Optical shop testing. 3rd ed. Hoboken, N.J.: Wiley-Interscience, c2007. ISBN 978-0471-48404-2.
- [3] WÜSTEFELD, Konstantin; EBBINGHAUS, Robin a WEICHERT, Frank. Learning to Segment Blob-like Objects by Image-Level Counting. Online. Applied Sciences. 2023, vol. 13, no. 22, s. 12219. ISSN 2076-3417. Dostupné z: <https://doi.org/10.3390/app132212219>. [cit. 2025-11-07].
- [4] STAŠÍK, Marek; KREDBA, Jan; NEČÁSEK, Jakub; LÉDL, Vít; FUCHS, Ulrike et al. Aspheric surface measurement by absolute wavelength scanning interferometry with model-based retrace error correction. Online. Optics Express. 2023, vol. 31, no. 8, s. 12449-12449. ISSN 1094-4087. Dostupné z: <https://doi.org/10.1364/oe.486133>. [cit. 2025-11-07].
- [5] STAŠÍK, Marek; PSOTA, Pavel; LÉDL, Vít a KREDBA, Jan. Subaperture stitching computation time optimization using a system of linear equations. Online. Applied

Optics. 2021, vol. 60, no. 27, s. 8556-8556. ISSN 1559-128X. Dostupné z:
<https://doi.org/10.1364/ao.433312>. [cit. 2025-11-07].
[6] OpenCV: OpenCV modules. n.d. <https://docs.opencv.org/4.x/>

Požadované náležitosti:

K obhajobě předložit zprávu o řešení projektu (15 stran, písmo 12 b) včetně anotace v českém a anglickém jazyce (viz [1]). Prezentace výsledků (max. 15 minut), ukázky realizovaného díla atp.

Termíny:

Odevzdání dokumentace a obhajoba podle harmonogramu výuky na FM.

Datum zadání: 12.2.2026

Podpis vedoucího projektu:

Marek
Stašík

Digitally signed by Marek Stašík
DN: CN=Marek Stašík, OU=TUL,
E=marek.stasik@tul.cz
Reason: I am the author of this
document
Location: Turnov
Date: 2026.02.13
19:36:50
+01'00'
Foxit PDF Reader Version:
2025.3.0

Vývoj algoritmu zarovnávání optických elementů s využitím obrazových dat ze zaměřovací kamery interferometru

Abstrakt

Tato semestrální práce se zabývá návrhem a implementací algoritmu pro detekci a identifikaci bodů na snímcích pořízených kamerou optického interferometru v zobrazovacím režimu. Práce se zaměřuje na zpracování obrazu a analýzu deformovaných bodů vznikajících vlivem interferenčních a optických jevů, kdy mohou být jednotlivé body částečně rozděleny do více oblastí. Navržený algoritmus využívá metody binarizace, filtrace a analýzy obrazu za pomoci knihovny OpenCV pro lokalizaci a určení průměrů detekovaných bodů. Implementace je realizována převážně v jazyce C# v prostředí Visual Studio 2022, přičemž MATLAB je využit jako doplňkový nástroj pro vizualizaci a testování vybraných metod zpracování obrazu. Výsledkem práce je funkční algoritmus schopný identifikovat významné body interferometrických snímků v rozumném čase.

Klíčová slova

optická interferometrie, interferometr, zobrazovací režim interferometru, zpracování obrazu, detekce bodů, parazitní odrazy, OpenCV, CLAHE, binarizace, morfologické operace, mediánová filtrace

Development of an algorithm for aligning optical elements using image data from an interferometer alignment camera

Abstract

This semester project focuses on the design and implementation of an algorithm capable of detecting and correctly identifying points captured by a camera of an optical interferometer operating in imaging mode. The work is focused on image processing and analysis of deformed points caused by optical and interferometric phenomena, where individual points may appear partially split into multiple regions. The proposed algorithm utilizes methods such as binarization, filtering, and image analysis using the OpenCV library for localization and diameter estimation of detected points. The implementation is primarily developed in the C# programming language within the Visual Studio 2022 environment, while MATLAB is used as a supplementary tool for visualization and testing of selected image processing methods. The result of this work is a functional algorithm capable of identifying significant points in interferometric images within a reasonable processing time.

Keywords

optical interferometry, interferometer, interferometer imaging mode, image processing, point detection, parasitic reflections, OpenCV, CLAHE, image binarization, morphological operations, median filtering

Obsah

Úvod	6
Teoretický základ.....	7
Interferometrie.....	7
Princip Fizeauova interferometru	7
Měření tvaru povrchu (interferenční režim).....	7
Měření vzdálenosti vzorku (zobrazovací režim)	8
Návrh řešení	9
Návrh řešení algoritmu	9
Realizace řešení	9
Zpracovávání snímku.....	10
Jednokanálový převod	10
Redukce šumu a zvýšení kontrastu	11
Prahování.....	12
Morfologické uzavření.....	13
Odstranění nežádoucích teček	15
Analýza bodů.....	16
Nalezení bodů.....	16
Výpočet radiusu.....	17
Výpočet souřadnic	17
Výsledky a testování.....	18
Závěr	19
Literatura.....	20

Úvod

Optické interferometry se používají pro velmi přesná měření povrchů a optických prvků. Aby měření dávalo správné výsledky, je nutné, aby byly jednotlivé části optické soustavy správně zarovnané. Pokud není vzorek nebo jiný optický element ve správné poloze, projeví se to na výsledném obrazu a může dojít ke zkreslení měření. Zarovnání je proto důležitou součástí práce s interferometrem.

V praxi se zarovnávání často provádí ručně podle obrazu ze zaměřovací kamery. Obsluha sleduje referenční body a odrazy a podle toho upravuje polohu vzorku. Tento postup je ale do určité míry subjektivní a závisí na zkušenosti konkrétní osoby. Zároveň může být časově náročný, zejména pokud je potřeba dosáhnout vyšší přesnosti. Z tohoto důvodu dává smysl pokusit se tento proces alespoň částečně automatizovat.

Tato práce se zabývá návrhem algoritmu, který dokáže zpracovávat obrazová data ze zaměřovací kamery interferometru a na jejich základě vyhodnotit polohu optických prvků. Konkrétně se zaměřuje na detekci referenčních a parazitních bodů v obraze a na rozpoznání odrazu od transparentního vzorku, například rovinného skla nebo čočky. Cílem je určit polohu a poloměr tohoto odrazu a následně vyhodnotit, jakým směrem a o jakou vzdálenost je potřeba vzorek posunout, aby došlo k jeho správnému zarovnání.

Algoritmus je implementován převážně v jazyce C# s využitím knihovny OpenCV v prostředí Visual Studio 2022. MATLAB je použit jako podpůrný nástroj pro zobrazení snímků a testování různých filtračních metod. Pro ověření funkčnosti jsou použity reálné snímky získané z interferometru, které jsou mezi sebou porovnávány a analyzovány.

V závěrečné části práce je věnována pozornost optimalizaci navrženého řešení, zejména z hlediska výpočetní náročnosti a odolnosti vůči šumu a parazitním jevům v obraze. Cílem je vytvořit algoritmus, který bude dostatečně spolehlivý a použitelný v reálných podmínkách měření.

Teoretický základ

Interferometrie

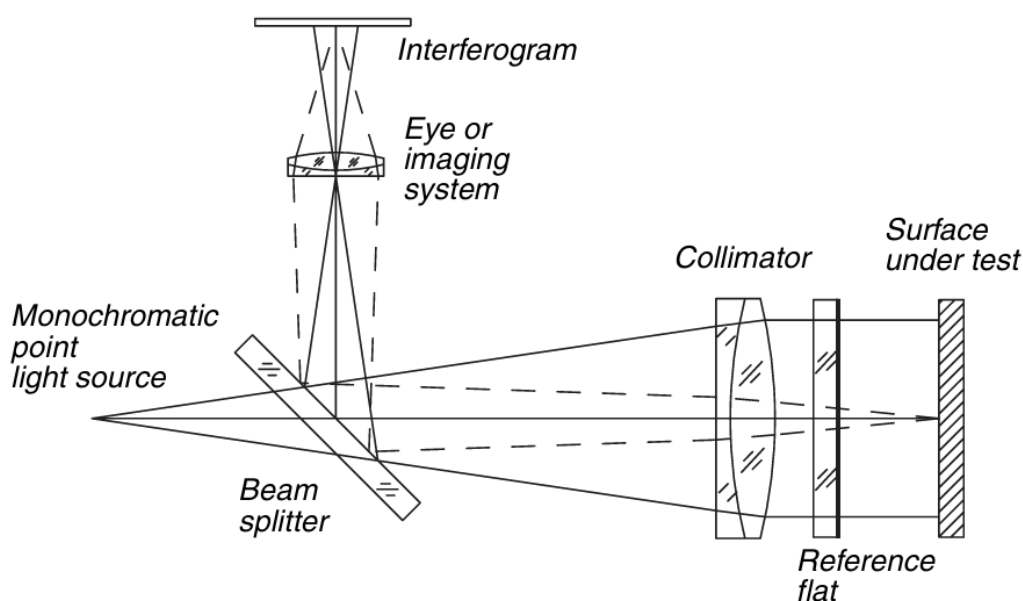
Interferometrie je vysoce přesná měřicí metoda využívající jevu interference světelných (nebo jiných) vln k měření délek, tvarů, indexů lomu či malých posunů. Princip spočívá v rozdělení koherentního svazku na dva, které urazí různé dráhy a následně spolu interferují, čímž vznikají interferenční proužky.

Princip Fizeauova interferometru

Princip Fizeauova interferometru (FI) se skládá ze sedmi částí. Infračervené světlo generované laserem (*monochromatic point light source*) dopadá na transparentní dělič paprsku (*beam splitter*), který rozdělí světlo směrem do kolimátoru (*collimator*) a do zobrazovací soustavy (*eye or imaging system*).

Kolimátor upraví svazek tak, aby byl rovnoběžný a dopadal kolmo na referenční destičku (*reference flat*). Na této destičce se část paprsku odrazí a část projde dále k testovanému povrchu (*surface under test*), od kterého se odrazí zpět.

Oba odražené paprsky (z referenční plochy i z testovaného povrchu) se vrací zpět přes kolimátor, přičemž si zachovávají svůj směr, a na děliči paprsky jsou odkloněny do zobrazovací soustavy. V prostoru, kde se tyto paprsky překrývají, dochází k jejich interferenci a zobrazovací soustava zaznamenává vzniklý interferogram.



Obrázek 1.1: Schéma Fizeauova interferometru[2]

Měření tvaru povrchu (interferenční režim)

Na interferogramu jsou při správném nastavení viditelné interferenční proužky[2]. Pokud je soustava správně zkalibrována a testovaný povrch ideálně rovinný, jsou proužky rovné a pravidelné. Odchylky od rovinnosti se projeví jejich deformací. Důležitým parametrem je kontrast proužků, tedy rozdíl mezi maximy a minimy intenzity.

Interference vzniká superpozicí elektrických polí obou paprsků podle vztahu $E = E_1 + E_2$. Protože oba paprsky pocházejí ze stejného zdroje, jsou koherentní a mají stabilní fázový rozdíl, který závisí na rozdílu optických drah. Právě tento rozdíl způsobuje vznik interferenčního obrazce.



Obrázek 1.2: Snímek z interferogramu pro měření tvaru povrchu vzorku (proužkový režim) [2]

Měření vzdálenosti vzorku (zobrazovací režim)



Obrázek 1.3: CF_B+0.02(snímek pořízen v laboratoři) [2]

Pro měření v bodovém režimu je potřeba nastavit interferometr jako zobrazovací režim. Výsledkem je snímek, který zobrazuje jednotlivé odrazy světla v optické soustavě jako bodové obrazy (PSF). Tyto body odpovídají referenčnímu paprsku, parazitním odrazům a odrazu od měřeného vzorku. Poloha těchto bodů v obraze je dána geometrickým uspořádáním soustavy a umožňuje identifikaci a určení polohy vzorku.

Na obrázku 2.1 je zachycen snímek pořízený v zobrazovacím režimu interferometru. Místo klasického interferogramu s proužky jsou zde patrné tři jasné bodové útvary, které odpovídají jednotlivým odrazům světla v optické soustavě. Každý bod představuje obraz jednoho koherentního paprsku – referenčního, parazitního a paprsku odraženého od měřeného vzorku. Body mají konečnou velikost v důsledku omezeného rozlišení optické soustavy a odpovídají bodové rozptylové funkci (PSF). Jejich poloha v obraze je dána geometrickým uspořádáním interferometru a umožňuje identifikaci jednotlivých složek signálu. Pro účely měření je klíčové určit bod odpovídající vzorku, což lze provést například sledováním jeho změny polohy při pohybu vzorku.

Návrh řešení

Návrh řešení algoritmu

Cílem práce je identifikace bodů ze snímků získaných kamerou interferometru nastaveného v zobrazovacím režimu. Hlavním výsledkem je analytické a grafické znázornění všech významných bodů nalezených ve snímku bez předem poskytnuté informace o jejich významu. Algoritmus je navržen tak, aby na základě analýzy poskytnutého snímku dokázal úspěšně lokalizovat jednotlivé body a určit jejich přibližný průměr v rozumném čase.

Realizace řešení

```
private Mat ProcessingImg(Mat img)
{
    Mat grayImg = ToGray(img);
    Mat contrastImg = Contrast(grayImg);
    Mat thresholdImg = Threshold(contrastImg);
    Mat morphImg = MorphClosing(thresholdImg);
    Mat cleanImg = removeDots(morphImg);

    grayImg.Dispose();
    contrastImg.Dispose();
    thresholdImg.Dispose();
    morphImg.Dispose();

    return cleanImg;
}
```

Obrázek 2.1: Návrh kódu pro zpracovávání snímku

Realizace řešení je provedena ve vývojovém prostředí Visual Studio 2022 od společnosti Microsoft v konzolové aplikaci v jazyce C#. Pro zpracování obrazu je využita knihovna OpenCV (Open Source Computer Vision Library). Kód na obrázku 2.1 představuje chronologické uspořádání modifikací obrazu. Algoritmus zpracovává snímek v následujících krocích:

1. Načtení snímku.
2. Převod snímku na jednokanálový černobílý obraz.
3. Redukce šumu a zvýšení kontrastu obrazu.
4. Převod obrazu na binární pomocí adaptivního prahování.
5. Výplň bodů pomocí morfologického uzavření.
6. Odstranění nežádoucích teček a šumu.
7. Výpočet souřadnic x , y a přibližného poloměru všech nalezených bodů.
8. Grafické zobrazení výsledků.

Zpracovávání snímku

Jednokanálový převod

```
private Mat ToGray(Mat img)
{
    if (img.Channels() == 1)
    {
        return img.Clone();
    }
    Mat g = new();
    Cv2.CvtColor(img, g, ColorConversionCodes.BGR2GRAY);
    return g;
}
```

Obrázek 3.1: Návrh kódu pro převod snímku na jednokanálový

Převod snímku na jednokanálový šedotónový obraz pomocí funkce *CvtColor* (Obrázek 3.1) snižuje následnou výpočetní náročnost a zjednodušuje další zpracování obrazu, jelikož jednotlivé pixely jsou reprezentovány pouze jasovou hodnotou v rozsahu 0 až 255, kde hodnota 0 představuje černou barvu a hodnota 255 bílou barvu. Následné algoritmy tak pracují pouze s jasovou složkou obrazu.

Redukce šumu a zvýšení kontrastu

```
private Mat EnhanceImage(Mat img)
{
    Mat den = new Mat();
    Cv2.MedianBlur(img, den, 3);

    Mat result = new Mat();

    var clahe = Cv2.CreateCLAHE(
        clipLimit: 20.0,
        tileGridSize: new Size(8, 8)
    );
    clahe.Apply(den, result);
    clahe.Dispose();
    den.Dispose();

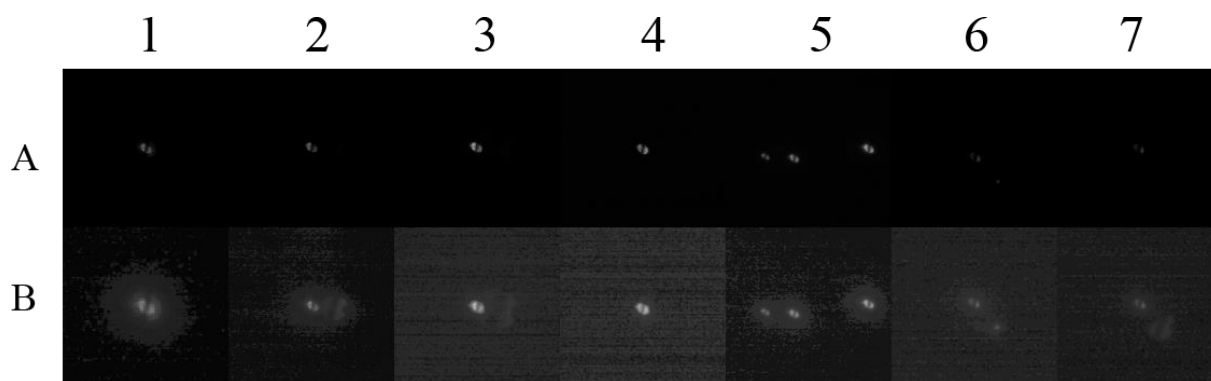
    return result;
}
```

Obrázek 3.2: Návrh kódu pro úpravu kontrastu snímku

Kontrast obrazu vyjadřuje rozdíl jasových hodnot mezi okolními pixely, přičemž vyšší kontrast zvýrazňuje jasnější pixely a potlačuje tmavší oblasti obrazu pomocí různých transformačních funkcí.

Pro zvýšení kontrastu byla použita transformační metoda *CLAHE* (*Contrast Limited Adaptive Histogram Equalization*). Tato metoda je založena na principu adaptivní úpravy kontrastu na základě hodnot získaných z histogramu obrazu. Tím dochází k lokálním úpravám kontrastu v jednotlivých částech obrazu namísto globální úpravy celého snímku. Implementaci této metody umožňuje funkce *CreateCLAHE*(*míra omezení kontrastu, rozdělení obrazu podle mřížky*). Funkce *clahe.Apply*(*vstup, výstup*) aplikuje *CLAHE*, provádí lokální vyrovnávání histogramu a zajišťuje plynulé přechody mezi jednotlivými oblastmi interpolace.[6]

Redukce šumu je realizována pomocí mediánového filtru za použití funkce *MedianBlur*(*vstup, výstup, velikost matice*). Princip spočívá v tom, že hodnota prostředního pixelu je nahrazena mediánem hodnot okolních pixelů v dané oblasti. Velikost masky byla zvolena na 3x3 pro jemné vyhlazení obrazu a částečné odstranění šumu.[6]



Obrázek 3.3: Ukázka modifikovaných snímků po zvýšení kontrastu odstranění šumu

Na obrázku 3.3 jsou v horní řadě zobrazeny jednokanálové snímky a ve spodní řadě snímky po aplikaci zvýšení kontrastu a mediánové filtrace. Body jsou na upravených snímcích výraznější, ovšem v obraze se stále nachází šum, jelikož použitý mediánový filtr nedokáže šum zcela odstranit bez ovlivnění viditelnosti zkoumaného vzorku (snímky B2, B3 a B7 – pravé body), jejichž viditelnost závisí na posunu po ose Z v interferometru.[3]

Prahování

```
private Mat Threshold(Mat img)
{
    Mat bw = new Mat();
    int pixelsTotal = (int)(img.Height * img.Width / 100 * 0.05);
    int whiteCount = 0;
    int k = 2;
    int th = 127;

    for (int i = 0; i < 8; i++)
    {
        Cv2.Threshold(img, bw, th, 255, ThresholdTypes.Binary);

        double thCompute = 127.5 / k;
        whiteCount = Cv2.CountNonZero(bw);

        if (whiteCount < pixelsTotal)
        {
            th = th - (int)thCompute;
        }
        else
        {
            th = th + (int)thCompute;
        }

        k = k * 2;
    }
}
```

```

    int whiteCountPost = 0;

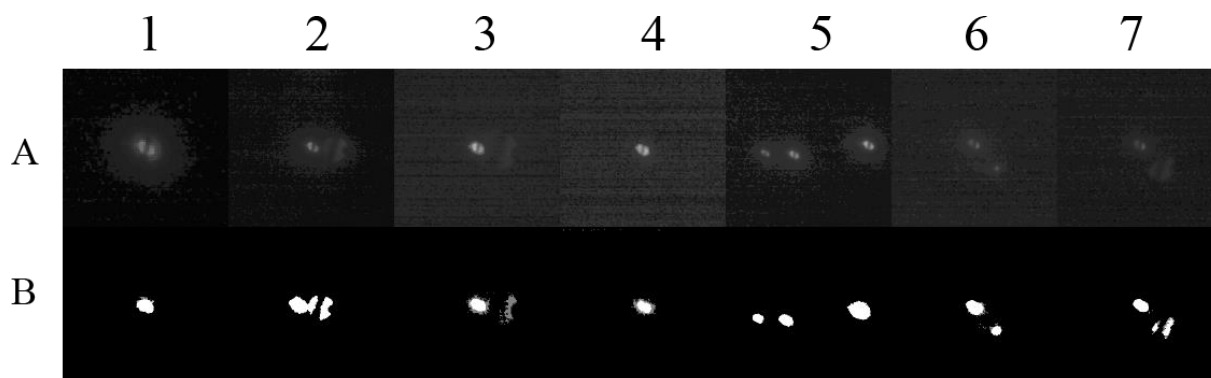
    for (int i = 0; i < 50; i++)
    {
        th -= 1;
        Cv2.Threshold(img, bw, th, 255, ThresholdTypes.Binary);
        whiteCountPost = Cv2.CountNonZero(bw);

        if (whiteCountPost - whiteCount > 50)
        {
            th += 1;
            Cv2.Threshold(img, bw, th, 255, ThresholdTypes.Binary);
            break;
        }
        else
        {
            whiteCount = whiteCountPost;
        }
    }
    return bw;
}

```

Obrázek 3.4: Návrh kódu adaptivního prahování

Adaptivní prahování je realizováno kombinací funkce *Cv2.Threshold(vstup, výstup, hodnota prahu, maximální hodnota prahu, typ binarizace)* a for cyklů. Princip prvního for cyklu spočívá v postupné změně hodnoty prahu tak, aby počet bílých pixelů odpovídal přibližně 0,05 % z celkového počtu pixelů snímku (prahování minima). Princip druhého for cyklu spočívá v postupné změně hodnoty prahu tak, aby počet bílých pixelů aktuálního snímku nepřekročil počet bílých pixelů o více než 0,01 % z celkového počtu pixelů snímku (prahování maxima).[6]



Obrázek 3.5: Ukázka modifikovaných snímků po binarizaci a prahování

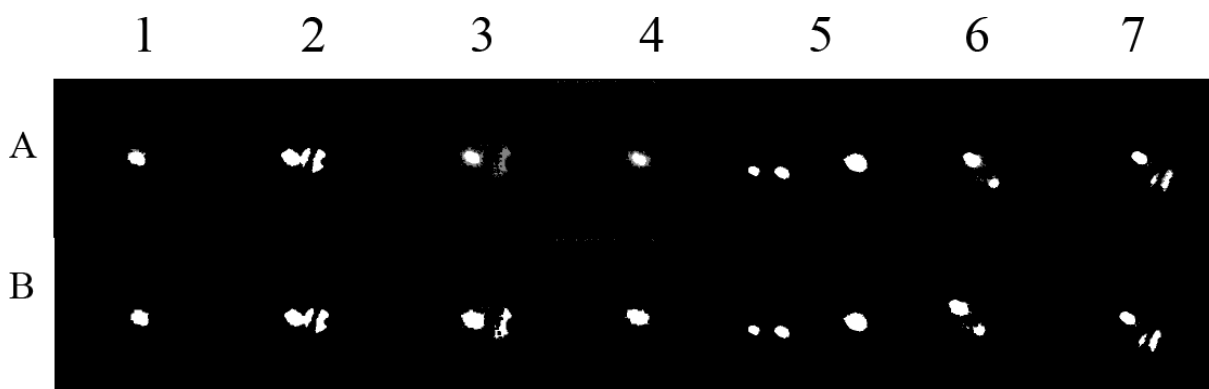
Na obrázku 3.5 jsou v horní řadě zobrazeny snímky po aplikaci zvýšení kontrastu a mediánové filtrace a ve spodní řadě snímky po aplikaci prahování a binarizace. Pokud je vzorek příliš vzdálený, může se jeho bodový obraz na snímku zobrazit nespojitě nebo částečně rozděleně (snímek B3 – pravý bod). Ve snímcích se stále nacházejí drobné bílé artefakty, které je potřeba odfiltrovat (například v horní části snímku B4).[4]

Morfologické uzavření

```
private Mat MorphClosing(Mat img)
{
    Mat imgClosing = new Mat();
    Mat kernel = Cv2.GetStructuringElement(MorphShapes.Rect, new Size(5, 5));
    Cv2.MorphologyEx(img, imgClosing, MorphTypes.Close, kernel);
    kernel.Dispose();
    return imgClosing;
}
```

Obrázek 3.6: Návrh kódu morfologického uzavření

Morfologické uzavření je metoda používaná pro vyhlazení a vyplnění objektů v binárním obrazu. Operace se skládá z dilatace následované erozí se stejnou maskou (kernel). Dilatace je proces využívající matici s lichým počtem řádků a sloupců, která postupně prochází obrazem a rozšiřuje bílé oblasti do okolních pixelů. Tím dochází k vyplnění menších mezer a částečnému spojení nespojitých oblastí objektu. Následná eroze omezuje nadměrné rozšíření objektů a obnovuje jejich přibližný původní tvar. Pokud po operaci dilatace následuje eroze se stejnou maskou, jedná se o morfologické uzavření. V implementaci byla použita funkce *Cv2.MorphologyEx(vstup, výstup, metoda morfologie, maska (kernel))*. [6]



Obrázek 3.7: Ukázka modifikovaných snímků po operaci morfologického uzavření

Na obrázku 3.7 jsou v horní řadě zobrazeny snímky po binarizaci a ve spodní řadě snímky po aplikaci morfologického uzavření. Rozdíly jsou patrné především u snímků B3, B4 a B7, kde došlo k vyplnění menších mezer a spojení nespojitých oblastí bodů. Na snímku A4 je v okolí bodu patrný drobný šum vzniklý v důsledku použité zaměřovací metody „cat’s eye“. Pro ostatní snímky byla použita zaměřovací metoda confocal. [5]

Odstranění nežádoucích teček

```
private Mat removeDots(Mat img)
{
    Mat labels = new Mat();
    Mat stats = new Mat();
    Mat centroids = new Mat();

    int n = Cv2.ConnectedComponentsWithStats(
        img, labels, stats, centroids
    );
    Mat clean = Mat.Zeros(img.Size(), MatType.CV_8U);

    for (int i = 1; i < n; i++)
    {
        int area = stats.At<int>(i, (int)ConnectedComponentsTypes.Area);

        if (area >= 20)
        {
            Mat mask = new Mat();
            Cv2.Compare(labels, i, mask, 0);
            clean.SetTo(255, mask);
            mask.Dispose();
        }
    }
    labels.Dispose();
    stats.Dispose();
    centroids.Dispose();
    return clean;
}
```

Obrázek 3.8: Návrh kódu pro odstranění nežádoucích teček

Na ukázce kódu je implementována metoda pro odstranění malých bodů a šumu z binárního obrazu pomocí analýzy propojených komponent. Nejprve jsou pomocí funkce *ConnectedComponentsWithStats* nalezeny všechny souvislé oblasti v obrazu, přičemž se pro každou oblast určí její plocha. Následně program prochází jednotlivé komponenty a zachovává pouze ty, jejichž plocha je větší nebo rovna hodnotě 20 pixelů. Menší oblasti jsou považovány za nežádoucí šum, a proto nejsou zahrnuty do výsledného obrazu. Výsledkem je vyčištěný binární obraz, ve kterém zůstávají pouze významné objekty a drobné artefakty jsou odstraněny.[6]



Obrázek 3.9: Ukázka modifikovaného snímku po odstranění nežádoucích bodů

Na obrázku 3.9 je náhled toho, jak se projeví změny snímku modifikovaného pomocí metody `removeDots` uvedené na obrázku 3.8. Okolní tečky jsou zcela odfiltrovány a snímek je připraven k další analýze.

Analýza bodů

Nalezení bodů

```
Cv2.FindContours(  
    bin,  
    out Point[][] contours,  
    out HierarchyIndex[] hierarchy,  
    RetrievalModes.External,  
    ContourApproximationModes.ApproxSimple  
);
```

Obrázek 4.1: Metoda pro nalezení spojitých bodů

```
for (int a = 0; a < contours[i].Length; a++)  
{  
    int y = contours[i][a].Y;  
    if (y < minY) minY = y;  
    if (y > maxY) maxY = y;  
}  
  
int minX = contours[i].Min(p => p.X);  
int maxX = contours[i].Max(p => p.X);  
  
int diameterX = maxX - minX;  
int diameterY = maxY - minY;  
  
Moments mom = Cv2.Moments(contours[i]);  
  
if (Math.Abs(mom.M00) < 1e-9) continue;  
int xImg = (int)Math.Round(mom.M10 / mom.M00);  
int yImg = (int)Math.Round(mom.M01 / mom.M00);  
if (diameterY > diameterX + devSpotY)  
{  
    xImg = xImg - diameterY / diameterX * 20;  
    diameterY = 30 * diameterY / diameterX;  
}  
int radiusPx = (int)Math.Round(diameterY / 2.0);  
  
if (radiusPx >= 2)  
{  
    foundCount++;  
    sp.Add(new Spot(xImg, yImg, radiusPx, 0f));  
}
```

Obrázek 4.2: Analýza lokalizace a poloměru bodu

Tato kapitola se zabývá analytickou analýzou všech bodů v upraveném snímku. Na obrázku 4.1 je zobrazena *metoda Cv2.FindContours(vstup, nalezené kontury (výstup), hierarchie kontur (výstup), typ kontur; metoda aproximace kontur)*, která slouží k nalezení obrysů objektů v binárním obrazu. Funkce prochází vstupní obraz a vyhledává souvislé oblasti tvořené bílými pixely, ze kterých následně vytváří seznam kontur reprezentujících hranice jednotlivých objektů. Výsledkem jsou pole bodů definující nalezené obrysy a informace o jejich hierarchii.

Výpočet průměru

Průměr bodů je určen ze dvou nejvzdálenějších pixelů konkrétního bodu po ose Y. V prvním for cyklu na obrázku 4.2 jsou získány nejnížší a nejvyšší hodnoty souřadnice Y ze všech pixelů nalezené kontury. Výpočet je prováděn po ose Y z důvodu častého horizontálního rozdělení bodových obrazů, kdy mohou být body vlivem optických a interferenčních jevů deformovány do více oblastí vedle sebe. Vertikální rozměr bodu tak poskytuje stabilnější aproximaci jeho průměru než výpočet po ose X.[2]

Výpočet souřadnic a korekce průměru

Výpočet souřadnic bodu X, Y je určen na základě těžiště detekované kontury pomocí obrazových momentů podle vztahů

$$x_c = \frac{M_{10}}{M_{00}}, \quad y_c = \frac{M_{01}}{M_{00}}$$

Kvůli interferenčním jevům může docházet k deformaci detekovaného bodu. Z tohoto důvodu je provedena dodatečná korekce polohy a velikosti bodu. Nejprve se z kontury určí její rozměry ve směru os X a Y, tedy rozdíl mezi maximální a minimální souřadnicí kontury. Pokud je výška kontury výrazně větší než její šířka, je poloha těžiště v ose X posunuta a hodnota průměru je upravena. Tato korekce je v kódu provedena v předposlední podmínce if.

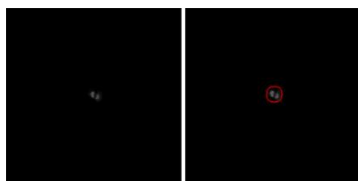
```
public class Spot
{
    private int coordX;
    private int coordY;
    private float coordZ;
    private int radius;

    Počet odkazů: 2
    public Spot(int coordX, int coordY, int radius, float coordZ)
    {
        this.coordX = coordX;
        this.coordY = coordY;
        this.radius = radius;
        this.coordZ = coordZ;
    }
}
```

Obrázek 4.3: Objekt bodu

Poslední podmínka if na obrázku 4.2 ukládá analytická data do listu objektů, který je znázorněn na obrázku 4.3.

Výsledky a testování



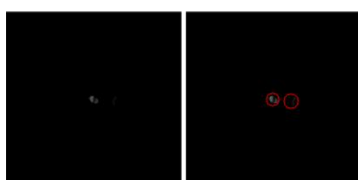
CE -16.bmp
X = 637
Y = 512
poloměr = 20



CF x+1.5 z-30.bmp
X = 639 X = 644
Y = 509 Y = 516
poloměr = 23 poloměr = 33



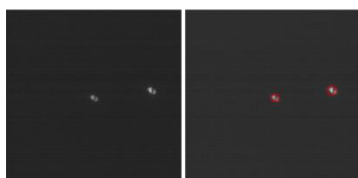
CF x+2 z-30.bmp
X = 639 X = 674
Y = 511 Y = 520
poloměr = 18 poloměr = 26



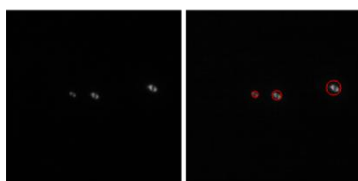
CF x+2 z-15.bmp
X = 681 X = 637
Y = 515 Y = 511
poloměr = 18 poloměr = 16



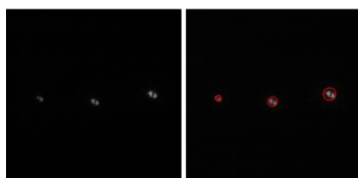
CF Z+1.bmp
X = 678 X = 638
Y = 553 Y = 511
poloměr = 8 poloměr = 12



Zygo flat 4: off_1.bmp
X = 604 X = 742
Y = 425 Y = 407
poloměr = 10 poloměr = 14



CF B+0.20.bmp
X = 640 X = 588 X = 777
Y = 505 Y = 502 Y = 487
poloměr = 12 poloměr = 9 poloměr = 18



CF B+0.50.bmp
X = 640 X = 510 X = 777
Y = 505 Y = 497 Y = 487
poloměr = 12 poloměr = 9 poloměr = 16



CF x+2 z-30.bmp
X = 639 X = 674
Y = 511 Y = 520
poloměr = 18 poloměr = 26

Navržený algoritmus byl testován na sadě snímků pořízených ze zaměřovací kamery interferometru. Testovací snímky obsahovaly různé počty bodů, rozdílnou intenzitu osvětlení (jasu) a odlišnou míru deformace bodových obrazů. U každého snímku byly detekovány významné body, pro které byla určena poloha ve směru os X a Y a jejich přibližný poloměr.

Výsledky detekce jsou znázorněny pomocí červených kružnic vykreslených do původního obrazu. Střed kružnice odpovídá vypočítanému těžišti detekované kontury a její poloměr odpovídá odhadnuté velikosti bodu. Z výsledků je patrné, že algoritmus je schopen detekovat body i v případech, kdy nejsou dokonale kruhové nebo jsou částečně deformované vlivem interferenčních jevů.

Horší výsledky mohou nastat u velmi slabě viditelných bodů nebo u bodů, které jsou výrazně rozděleny do více částí. V těchto případech může dojít k nepřesnému určení těžiště nebo poloměru. Přesto je navržený postup pro testované snímky použitelný a umožňuje automatizovaně určit základní parametry bodů potřebné pro další vyhodnocení zarovnání optických elementů.

Závěr

Cílem semestrální práce bylo vytvoření systému pro detekci optických bodů ze snímků získaných kamerovým systémem při ustavování optických prvků. Pro dosažení tohoto cíle bylo nutné navrhnout vhodný postup zpracování obrazu a implementovat algoritmy pro detekci polohy a velikosti jednotlivých bodů.

V první části práce byly použity metody předzpracování obrazu. Snímky byly převedeny do odstínu šedi, následně byl zvýšen kontrast pomocí metody *CLAHE* a odstraněn šum mediánovým filtrem. Poté bylo provedeno binární prahování a morfologické operace, které umožnily lepší detekci kontur objektů.

Pro určení polohy bodů byly využity obrazové momenty, pomocí kterých bylo vypočítáno těžiště jednotlivých kontur. Z důvodu deformací způsobených interferenčními jevy byla implementována také korekce polohy a velikosti detekovaných bodů. Tato úprava zlepšila přesnost výsledné lokalizace.

Algoritmus byl testován na různých typech snímků obsahujících jeden i více optických bodů. Výsledky ukázaly, že navržené řešení je schopné spolehlivě určit polohu bodů i v případě deformovaných nebo méně kontrastních obrazů.

Výsledkem práce je aplikace implementovaná v jazyce C# s využitím knihovny OpenCV, která umožňuje automatické zpracování snímků a vizualizaci detekovaných bodů.

V budoucnu by bylo možné práci rozšířit například o přesnější subpixelovou lokalizaci bodů, podporu online zpracování obrazu nebo využití GPU akcelerace pro rychlejší výpočty.

Použitá literatura

- [1] PLÍVA, Z., J. DRÁBKOVÁ, J. KOPRNICKÝ a L. PETRŽÍLKA. *Metodika zpracování bakalářských a diplomových prací*. 3. upravené vydání. Liberec: Technická univerzita v Liberci, FM, 2019. ISBN 978-80-7494-455-0. Dostupné z: doi:10.15240/tul/002/978-80-7494-455-0.
- [2] Optical shop testing. 3rd ed. Hoboken, N.J.: Wiley-Interscience, c2007. ISBN 978-0471-48404-2.
- [3] WÜSTEFELD, Konstantin; EBBINGHAUS, Robin a WEICHERT, Frank. Learning to Segment Blob-like Objects by Image-Level Counting. Online. Applied Sciences. 2023, vol. 13, no. 22, s. 12219. ISSN 2076-3417. Dostupné z: <https://doi.org/10.3390/app132212219>. [cit. 2025-11-07].
- [4] STAŠÍK, Marek; KREDBA, Jan; NEČÁSEK, Jakub; LÉDL, Vít; FUCHS, Ulrike et al. Aspheric surface measurement by absolute wavelength scanning interferometry with model-based retrace error correction. Online. Optics Express. 2023, vol. 31, no. 8, s. 12449-12449. ISSN 1094-4087. Dostupné z: <https://doi.org/10.1364/oe.486133>. [cit. 2025-11-07].
- [5] STAŠÍK, Marek; PSOTA, Pavel; LÉDL, Vít a KREDBA, Jan. Subaperture stitching computation time optimization using a system of linear equations. Online. Applied Optics. 2021, vol. 60, no. 27, s. 8556-8556. ISSN 1559-128X. Dostupné z: <https://doi.org/10.1364/ao.433312>. [cit. 2025-11-07].
- [6] OpenCV: OpenCV modules. n.d. <https://docs.opencv.org/4.x/>